

Validating data before training

Collected movie data arrives as strings, with repeats and "N/A". Check it before a model sees it.

THE QUESTION

The API calls all succeeded. Is the collected data fit to train on, and how would you know?

THE INTUITION

Treat raw data as suspect. Look at it with cheap command-line tools, write the rules for a valid record as a schema, and check each record at the door. Failures are set aside with a reason.

KEY TAKEAWAY

Inspect before you trust, validate before you use. Models train on bad data without complaint, so count what you reject and coerce.

raw data → inspect: wc, head, jq → profile: csvstat → validate: schema → clean → valid rows + rejects

1 Collected is not the same as usable

title	year	runtime	rating	boxoffice	
Inception	2010	148 min	8.8	\$292576195	duplicate row
Avatar	2009	162 min	7.9	\$2923706026	missing: N/A
The Room	2003	99 min	3.9	N/A	number + unit
Inception	2010	148 min	8.8	\$292576195	\$ in a number
Tenet	N/A	150 min	7.3	N/A	

loud: model.fit raises
ValueError: '148 min'

silent: coerce year, rating; dropna
95 → 93 rows, no warning

The first rows of the demo's movies.csv. A string such as "148 min" makes model.fit raise, which is the lucky case. Coercing to numbers and calling dropna() is quieter and worse: rows vanish with no warning. Print row counts after every step.

2 Six ways data can be wrong

Completeness	is it all there?	Validity	does it obey the rules?
Year "N/A" for Tenet		rating 15.0 on a 0 to 10 scale	
Uniqueness	any repeats?	Consistency	same thing, same form?
Spider-Man appears 3 times		"USA" vs "United States"	
Accuracy	is it true?	Timeliness	is it current?
year 2099 for a 1999 film		2019 prices used in 2024	

Missing has four forms: "", null, a sentinel ("N/A", -1, 9999) and an absent key. Add wrong types ("2010"), mixed formats (\$292,576,195 and \$292.5M), broken encodings and schema violations. The 1-10-100 rule: checking at entry costs 1, fixing later 10, acting on bad data 100.

3 Look before you load

```
$ wc -l movies.csv
96 movies.csv                      1 header + 95 rows

$ grep -c "N/A" movies.csv
8                                  lines, not cells

$ grep -c ",," movies.csv
1                                  an empty field mid-row

$ cut -d',' -f1 movies.csv | sort | uniq -c
3 Spider-Man, 2 Inception, 2 The Matrix

uniq sees neighbours only: sort first
```

Real outputs on the demo file. file -i (-I on macOS) names the encoding, head and tail show both ends, tail -n +2 skips the header, sort -t', ' -k2 -nr sorts on column 2. grep -c counts lines, not cells; -n adds line numbers, -v inverts, -i ignores case.

4 A comma inside quotes breaks split and cut

one line of movies.csv, cut at every comma:

```
Inception 2010 148 min 8.8 $292576195 "Action Adventure Sci-Fi" PG-13
```

→ 9 fields; the header has 7

fields per line (all 96 lines)

split(',') 7 fields	19
split(',') 8 fields	24
split(',') 9 fields	53
csv.reader 7 fields	96

A field with a comma, quote or newline is quoted, and inner quotes are doubled: "Say ""Hello""". Parse with csv.reader, pandas or csvkit, never split(",").

5 jq filters JSON one step at a time

.	.[]	select(.Year == "N/A")	.Title
24-item array	24 objects	1 object	"Tenet"

jq '{total: length, null_titles: ..., ...}' gives

```
{
  "total": 24,
  "null_titles": 2,
  "na_years": 1,
  "na_boxoffice": 3
}
```

"Unknown Movie" has "Rating", not "imdbRating"

```
jq '.[ ] | select(.Year == "N/A") | .Title'
jq '.[ ] | select(.Title == "") | length'
jq -r '.[ ] | [.Title, .Year] | @csv'
```

.[] walks an array, | pipes, {name: .Title} builds an object, .rating // "N/A" sets a default, has("Budget") tests a key, tonumber converts, -r drops quotes. Also unique, sort_by(.Year), group_by(.Rated).

6 Profile every column before trusting it

column	N/A or ""	range / distinct	look closer
title	1	91 of 95	3 titles repeat
year	1	-500 to 2030	negative year
runtime	1	45 to 585 min	"abc min"
rating	1	3.9 to 15.0	above 10
boxoffice	8	text with \$	-50000000
rated	0	9 values	"NR", "Not Rated", "XX"

csvstat gives this per column (type, nulls, distinct, range, most common); csvlook prints a table, csvcut -n lists columns, csvgrep filters rows, csvsql --query runs SQL on a CSV. In pandas, read_csv already reads "" and N/A as NaN, so year loads as a float; "Not Rated" stays text.

Write the rules down, check every record

JSON Schema and Pydantic check records against stated rules; a pipeline keeps the valid ones and logs the rest.

7 A JSON Schema is a contract for one record

```
{
  "title": "",
  "year": 1500,
  "rating": 15.0,
  "genres": [],
  "rated": "INVALID"
}
```

minLength	"" is shorter than 1
minimum	1500 is below 1880
maximum	15.0 is above 10
minItems	[] has under 1 item
enum	"INVALID" is not in the enum

iter_errors → 5 errors at once; validate() stops at the first

```
schema = {"type": "object",
  "required": ["title", "year", "genres"],
  "properties": {
    "year": {"type": "integer",
      "minimum": 1880,
      "maximum": 2030},
    "rated": {"enum": ["G", "PG", "R"]}}
validate(instance=movie, schema=schema)
```

It is plain JSON, so an API and clients in any language share one file. Also: ["number", "null"], pattern, format, items, uniqueItems. Of the demo's batch of five, two fail: an empty title and the year 2050.

Draft202012Validator(schema).iter_errors(movie) yields every error, as in the figure.

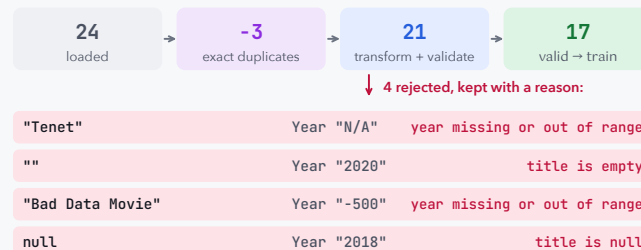
8 Pydantic checks when the object is built

input	JSON Schema	plain type Pydantic	with Field(...) Pydantic
year="2010"	rejected	2010	2010
year="not a year"	rejected	error	error
title=""	rejected	""	error
rating=15.0	rejected	15.0	error

```
class Movie(BaseModel):
    title: str = Field(min_length=1)
    year: int = Field(ge=1888, le=2030)
    rating: float | None = None
m = Movie(**raw) # ValidationError if bad
```

After creation every field has its type, so `m.year + 1` is safe. Pydantic converts "2010" where JSON Schema refuses it; strict mode refuses too. A plain str accepts "", so add `min_length`. `e.errors()` lists every bad field, `@field_validator` adds custom rules, `model_dump()` returns a dict. Use Pydantic inside Python, JSON Schema across languages.

9 Transform, validate, keep the rejects



The demo pipeline on `movies.json`: transform turns "\$292,576,195" into 292576195 and "148 min" into 148, then `CleanMovie` checks the result.

```
ok, bad = [], []
for i, raw in enumerate(data):
    try:
        m = CleanMovie(**transform(raw))
        ok.append(m)
    except ValidationError as e:
        bad.append((i, raw, str(e)))
```

Check at the boundary, before bad rows spread; never except: pass.

10 Rules catch only what they state

Unknown Movie	revenue 999,999,999,999	only ge=0 on revenue
Unknown Movie	key "Rating" → rating None	typo in a key name
Joker	"invalid" → rating None	cleaner hid the error
Spider-Man x2	7.4 vs 7.3	near duplicate kept

All four were valid in the same run. A cleaner that maps bad input to None hides the error, so keep checking and fixing apart and count the Nones. Test the rules at the edges: 1879, 1880, 2030, 2031, "2010", None.

11 Text is bytes plus an encoding

A m é l i e text
 41 6D C3 A9 6C 69 65 UTF-8 bytes
 A m Ã © l i e read as Latin-1
 "Amélie" saved as UTF-8, opened as Latin-1 → "AmÃ©lie"
 line ends: LF \n = 0A (Unix, macOS) · CRLF \r\n = 0D 0A (Windows)

Name it when reading: `open(p, encoding="utf-8")`. Convert with `iconv -f ISO-8859-1 -t UTF-8; strip \r` with `dos2unix`.

12 Pause and think

`uniq -d` finds no repeats in an unsorted file. Why? It compares neighbouring lines only; sort first.

JSON Schema rejects "2010", Pydantic accepts it. Which is right? Both; use strict mode when a string year means a broken source.

17 of 21 passed. Ready to train? Not yet: a 999-billion revenue, a lost rating and two Spider-Man rows got through.

look first · a schema for every record · coerce on purpose, count the NaNs · log rejects with reasons

Source: lecture "Data validation and quality" with its demos (`movies.csv`, `movies.json`, `movie_schema.json`) and lab, nipunbatra.github.io/stt-ai-teaching. Collecting the data is on the HTTP and APIs sheet and the scraping sheet.